

Data Structures

Dr. Tayyaba Khushid

Department of Cyber Security,
Dawood University of Engineering and Technology (DUET).

August 27, 2026

Course Outline

Abstract data types, complexity analysis, Big Oh notation, Stacks (linked lists and array implementations), Recursion and analyzing recursive algorithms, divide and conquer algorithms, Sorting algorithms (selection, insertion, merge, quick, bubble, heap, shell, radix, bucket), queue, dequeuer, priority queues (linked and array implementations of queues), linked list its various types, sorted linked list, searching an unsorted array, binary search for sorted arrays, hashing and indexing, open addressing and chaining, trees and tree traversals, binary search trees, heaps, M-way tress, balanced trees, graphs, breadth-first and depth-first traversal, topological order, shortest path, adjacency matrix and adjacency list implementations, memory management and garbage collection

Introduction

Data structures and algorithms (DSA) are two important aspects of any programming language. Every programming language has its own data structures and different types of algorithms to handle these data structures.

Data structure and Algorithm Definition

Definition

Data Structures are used to organize and store data to use it in an effective way when performing data operations.

Definition

An algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output.

Basic types of Data Structures

As we have discussed above, anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc, all are data structures. They are known as Primitive Data Structures.

Then we also have some complex Data Structures, which are used to store large and connected data. Some example of Abstract Data Structure are :

Basic types of Data Structures

- Linked List
- Tree
- Graph
- Stack, Queue etc.

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required. We will look into these data structures in more details in our later lessons.

Basic types of Data Structures

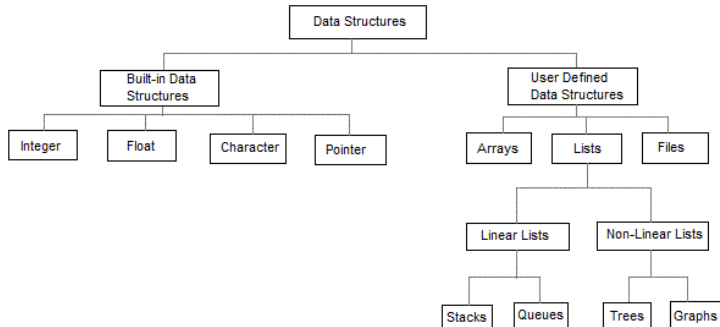


Figure: Introduction to Data structure

Basic types of Data Structures

The data structures can also be classified on the basis of the following characteristics:

- Linear: In Linear data structures, the data items are arranged in a linear sequence. Example: Array
- Non-Linear In Non-Linear data structures, the data items are not in sequence. Example: Tree, Graph
- Homogeneous In homogeneous data structures, all the elements are of same type. Example: Array

Frame Title

- Non-Homogeneous In Non-Homogeneous data structure, the elements may or may not be of the same type. Example: Structures
- Static Static data structures are those whose sizes and structures associated memory locations are fixed, at compile time. Example: Array
- Dynamic Dynamic structures are those which expands or shrinks depending upon the program need and its execution. Also, their associated memory locations changes. Example: Linked List created using pointers

Algorithm

An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task. Algorithm is not the complete code or program, it is just the core logic(solution) of a problem, which can be expressed either as an informal high level description as pseudocode or using a flowchart.

Every Algorithm must satisfy the following properties:

- Input- There should be 0 or more inputs supplied externally to the algorithm.
- Output- There should be atleast 1 output obtained.
- Definiteness- Every step of the algorithm should be clear and well defined.
- Finiteness- The algorithm should have finite number of steps.
- Correctness- Every step of the algorithm must generate a correct output.

Introduction

An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured on the basis of following properties :

- Time Complexity
- Space Complexity

Space Complexity

Its the amount of memory space required by the algorithm, during the course of its execution. Space complexity must be taken seriously for multi-user systems and in situations where limited memory is available.

An algorithm generally requires space for following components :

- **Instruction Space:** Its the space required to store the executable version of the program. This space is fixed, but varies depending upon the number of lines of code in the program.
- **Data Space:** Its the space required to store all the constants and variables(including temporary variables) value.
- **Environment Space:** Its the space required to store the environment information needed to resume the suspended function.

Time Complexity

Time Complexity is a way to represent the amount of time required by the program to run till its completion. It's generally a good practice to try to keep the time required minimum, so that our algorithm completes its execution in the minimum time possible.

Applications of Data Structures Algorithms (DSA)

- Search Algorithm to search an item in a data structure.
- Sort Algorithm to sort items in a certain order.
- Insert Algorithm to insert item in a data structure.
- Update Algorithm to update an existing item in a data structure.
- Delete Algorithm to delete an existing item from a data structure.

Types of Data Structures

The different type of data structures are:

- Array Data Structure
- String Data Structure
- Stack Data Structure
- Queue Data Structure
- Matrix/Grid Data Structure
- Tree Data Structure

Types of Algorithms

- Searching Algorithms
- Sorting Algorithms
- Greedy Algorithms
- Mathematical Algorithms
- Geometric Algorithms